Data Exchange via Multiplexed Color QR Codes on Mobile Devices

Sin Rong Toh
School of Computer Engineering
Nanyang Technological University
Singapore
stoh003@e.ntu.edu.sg

Weihan Goh
Infocomm Technology Cluster
Singapore Institute of Technology
Singapore
Weihan.Goh@Singaporetech.edu.sg

Chai Kiat Yeo
School of Computer Engineering
Nanyang Technological University
Singapore
asckyeo@ntu.edu.sg

***Abstract*—This paper presents a new way of exchanging data such as images and text files via mobile devices in the absence of existing wireless technologies, namely WiFi and Cellular Data Network in a personal area network. It does not incur the overhead of Bluetooth pairing and discovery. This application, ColorQR, sends data through the manipulation of colors into multiplexed QR codes. Data is encoded into Red-, Blue- and Green- color QR codes and are multiplexed to form multiple multi-color QR codes that will be flashed on the screen of the sender's device. The receiver, in the form of a camera of a mobile device, will record the flashing images and convert the images into RGB channels using grayscale conversion. Thus one image will be split into 3 sets of black and white QR codes. Results show that ColorQR is highly feasible and it can transmit data of varying sizes without limitation.**

Keywords—Multiplexed QR code; 3D QR code, data exchange in PAN

I. INTRODUCTION

The use of WiFi, Bluetooth and Cellular Network has made exchanging files on mobile devices easy and convenient. They offer a reliable service when signal or connection is stable and available. However, sometimes this is not the case. When people need to exchange files and they are just next to each other but situated in a place with no WiFi or when the user's mobile phone does not have Cellular Network connection, it is not possible to exchange files. Moreover, sending a file through Cellular Network may incur cost compared to using WiFi hotspots.

For Bluetooth, if many devices are in the same piconet, it can lead to congestion and poor network performance, as the signals collide and information has to be resent [1]. Moreover, when using Bluetooth, users need to identify and pair with each other's mobile devices before file exchange is possible. This discovery takes time.

If all these three technologies are not available to a device, exchanging data is also not possible. Thus, a new communication channel could be created which does not use existing radio communication but a property that most mobile devices has, which is the use of the camera.

This paper proposes an Android appplication called ColorQR which lets users use their mobile devices to send image and text files through a lightweight method that uses colors and image sensors. The colors are simply a series of multiplexed Red, Blue and Green QR codes that will flash on the sender's screen and the receiver will use their mobile device camera to record the images flashing on the sender's screen. (Note: Each multiplexed QR code that is being flashed will be referred to as an image. A frame refers to each Red, Blue or Green QR code). This method of transmission is useful in the absence of Internet connectivity when users are next to each other. ColorQR is able to transmit more data and at a higher throughput than current QR code.

II. RELATED WORKS

The QR code was originally invented to track vehicles during manufacturing. However, QR codes are now extended to be used in commercial settings. Many mobile users have used QR codes to send texts, open a Uniform Resource Locator (URL), or to send e-mail or text messages. The technology has since become one of the most commonly used two-dimensional (2D) barcodes.

The current design of QR code is made up of black and white squares and it can store variable sized data. A Version 1 QR Code is a 21×21 array of data elements and can hold up to 25 alphanumeric data with the array increasing in size by 4 modules (25×25 for Version 2) for each increase in version number. The largest standard QR Code is a Version 40 symbol that is 177×177 modules in size and can hold up 4296 characters of alphanumeric data.

Tables I and II show the version and the corresponding details of the current QR codes and the multiplexed QR codes used in ColorQR respectively. One multiplexed QR code can contain three times more data than the current QR code for each version.

There are 4 error correction levels used for QR code. The QR code can continue to function even when a part of the QR code image is obscured, defaced or removed: Level L – up to 7% loss, Level M – up to 15% loss, Level Q – up to 25% loss, Level H – up to 30% loss [2]. In ColorQR, Level L is used as it is highly unlikely that the QR codes will be damaged. If the error correction level increases, we will need to compensate for the increase data load and the QR code will be denser.

TABLE I. VERSION INFORMATION FOR EXISTING QR CODE

Version	Module	ECC Level	# of Encoded Alphanumeric Values
10	57×57	L	395
11	61×61	L	468
12	65×65	L	535

TABLE II. VERSION INFORMATION FOR MULTIPLEXED QR CODE

Version	Module	ECC Level	# of Encoded Alphanumeric Values
10	57×57	L	1185
11	61×61	L	1404
12	65×65	L	1605

With the prevalent use of QR code in daily life, the storage capacity can be further expanded by multiplexing primary and secondary colors into it and utilizing it as a data exchange platform. The following are some previous works done to enable sending data through colors and increasing QR code storage size.

PicapiCamera, developed by Casio, is the world's first iPhone app which uses visible light communication technology. To send and receive data via the app, the message to be sent is encoded using red, green and blue flashing lights. The receiver points his iPhone at the flashing lights and the data is transferred. As the amount of data that can be transferred via the sequence of flashing lights is only a single 8-bit word, an ID number in conjunction with GPS data is used to download the relevant contents from a cloud server [3]. This method of sending and receiving data incorporates GPS data and cloud server due to the fact that it uses only 3 colors in a serial manner and thus has limited message size.

Using a simple method of keyboard symbols, Harish and Gurav proposed a method of encoding each pattern and representing each module in QR code with special symbols [4]. At the receiving end, this QR code with special symbols (that was multiplexed) is decoded to return the number of QR code patterns that was multiplexed.

2D Datamatrix barcodes is widely used in printed media and in the food industry which allows the media / foodstuff to be tracked easily. Langlotz and Bimber encoded whole dataset which is split into smaller portions, into a series of multiplexed RGB 2D Datamatrix barcodes. The decoding method is similar to the proposed ColorQR, which is to convert each RGB channel to grayscale. The transmission rate for that project is 23 characters per second [5].

Hao et. al. also proposed a method similar to [5] using Datamatrix barcodes [6].

2D codes used together with GPS and / or the Internet can enable users to discover more information by scanning the code and then accessing information hinted by the code. Owing to the small capacity of 2D codes, Internet connectivity is typically still needed when working with 2D codes. Microsoft developed the High Capacity Color Barcode Technology which enables higher density storage of data on analog printed media. The format achieves this by using a different barcode symbol shape in combination with multiple colors per symbol [7]. However, as it is only designed for static printed use, it cannot flexibly increase its data transmission size and users will still have to connect to the Internet to see more information. Therefore, it is more often used for commercial purposes and not for file transmission.

Rohs presented a system that turns camera phones into mobile sensors for black and white 2D visual codes. It aims to integrate phone movement detection features in a number of ways to interact with individual items on a large-scale wall display [8]. Basically, the camera can point at different angles on a 2D code and will get different information from it. This application also needs Internet capable devices to be able to display certain information and is so far tested with large scale displays instead of between mobile devices.

Werner and Schönfeld proposed a technique which can be used to increase the amount of information contained in a matrix barcode in a backward compatible way. This enables applications to fully utilize the widely used QR Codes or Data Matrix Codes for service discovery or basic service, while being able to transmit much more information during one scan for advanced applications [9]. It uses a similar method as the proposed ColorQR as in it combines two layers of QR code into one.

Bulan et. al. discovered a method that enabled printing of two colors at the same spatial location, and hence (nearly) doubles the capacity in comparison with encoding methods based on a single colorant channel. This method can significantly outperform the state of the art 2D barcodes achieving a higher embedding rate [10].

André and Ferreira also exploited color multiplexing of QR code. However, the display of the code is static and is not received in video mode [11].

III. IMPLEMENTATION

To receive the data sent, users need to point the camera's center on the QR code shown in Figure 1 and start capturing it as soon as it appears. This QR code is a sequence of '0' characters and acts as a 'Start' frame to prepare the user to capture the start of the QR code frames. The sent data takes the form of multiplexed QR codes flashing at 0.5s each, at the end of which, a 'Stop' QR code, similar to the 'Start' one will appear to signal the user to stop recording. The data in the Start / Stop QR code ('0' characters) is decoded and ignored at the receiver. Checksum is added to the data to be sent for error detection.

A. Text File Transfer

Figure 2 shows the overview of the encoding and decoding process for text file transfer.

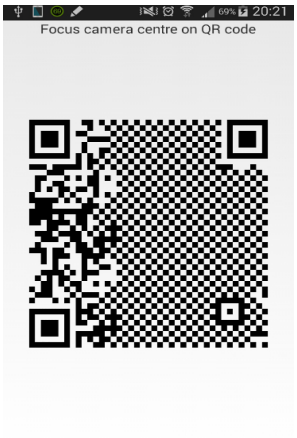


Fig. 1. Start / Stop QR code.

1) Encoding

First, ColorQR splits the text file into almost equal chunks, each about a third of the file size (e.g. a text file of 900 bytes will have each chunk of 300 bytes). These chunks are then converted to QR code. The maximum size of each QR code is 500 bytes. Three QR codes (Red QR, Green QR and Blue QR) are multiplexed to represent one image. When they overlap with each other, they produce secondary colors. Thus one image consists of a mixture of all primary colors and secondary colors. A total of 8 colors are used including black and white. Refer to Table III for the mixture of primary colors. When the primary colors overlap one another, white results. Thus an invalid QR code is shown in Figure 3 for such a scenario. The colors are then inverted to produce the "Position" and "Alignment" squares (i.e. the black squares at the 3 corners of the QR codes shown in Figure 3). Figure 3 also shows the Red, Green and Blue QR codes that are being orthogonally color multiplexed into a single QR code. The Red, Green and Blue QR codes are multiplexed by using Android's *PorterDuffXfermode*.

TABLE III. COLOR COMBINATION CHART

Color 1	Color 2	Color 3	Result
Red	Green		Yellow
Green	Blue		Cyan
Red	Blue		Magenta
Red	Green	Blue	White

TABLE IV. COLOR INVERSION CHART

Color	⟷	Inverse
Red	⟷	Cyan
Green	⟷	Magenta
Blue	⟷	Yellow
White	⟷	Black

2) Decoding

The receiver's mobile device camera is used to record all the images in sequence. The app then extracts the images at 5 frames per 0.5 s (i.e. for a 3-second video, 30 frames will be extracted). The 5 frames will contain the same QR code information but they may differ in the completeness of the QR code. The purpose of using 5 frames is for redundancy in case the first frames cannot be decoded due to refresh from the screen or other errors. Thus, ultimately, only 1 frame per 0.5 s will be used to decode the output as each image is flashed on the sender's screen for 0.5s.

At the start, the first image is extracted and split to Red, Green and Blue 'channels' using grayscale conversion as shown in Figure 3. The decoding results in 3 black and white QR codes for one image which correspond to their original color QR code before the color inversion. This is done using OpenCV API [12]. Table V shows how the separation of the color frames is done.

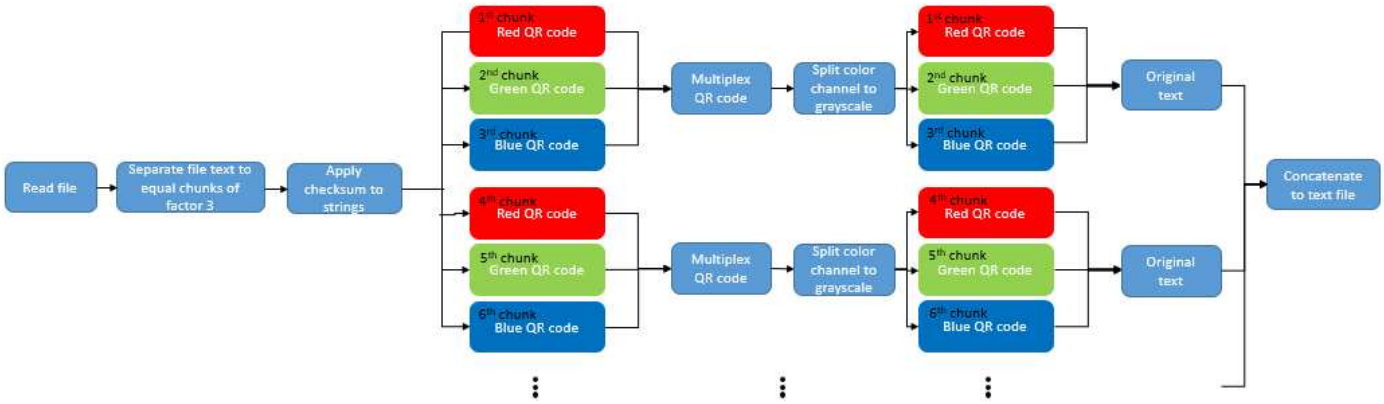


Fig. 2. Overview of the Encoding and Decoding of Text File in ColorQR.

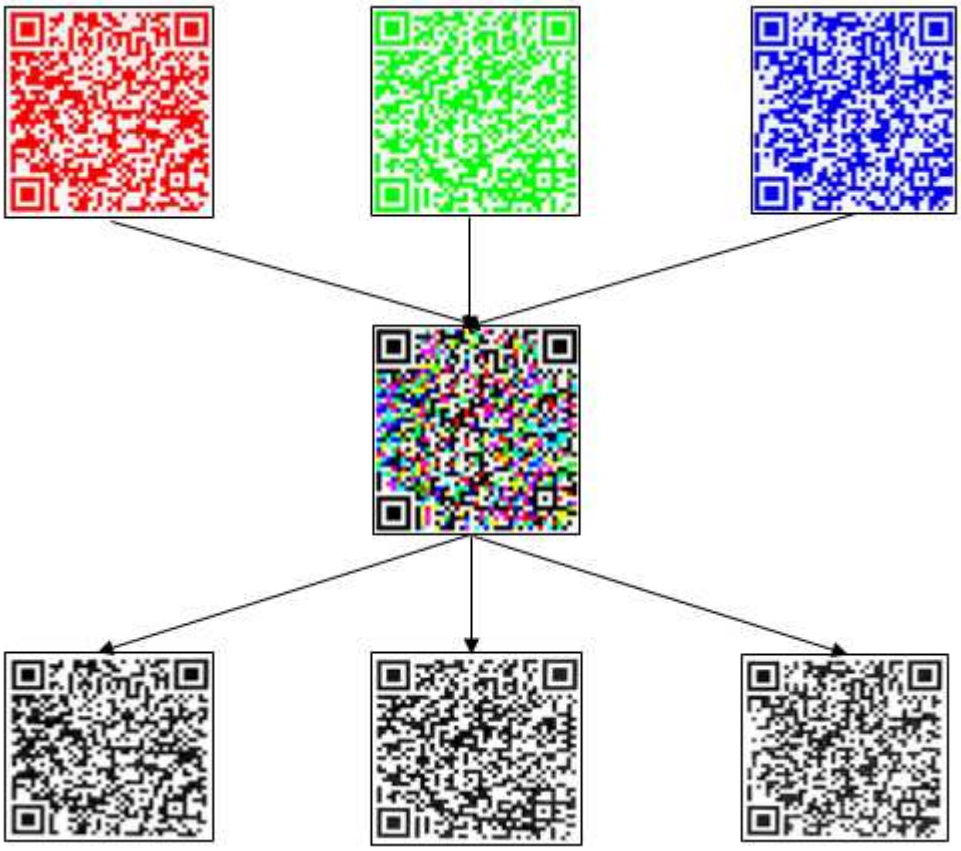


Fig. 3. Multiplexing and Demultiplexing of QR codes. The 3 Colored QR codes are multiplexed into a single QR code during the encoding at the sender. The single multiplexed QR code is color inverted and demultiplexed during the decoding at the receiver.

TABLE V. SEPARATION OF COLOR CHANNELS

Original Color	⟷	Inverse	Hex Representation (0xRRGGBB)	Black Pixels	White Pixels
Red	⟷	Cyan	0x**00**FFFF	Red	Green & Blue
Green	⟷	Magenta	0xFF**00**FF	Green	Red & Blue
Blue	⟷	Yellow	0xFFFF**00**	Blue	Red & Green
Cyan	⟷	Red	0xFF**0000**	Green & Blue	Red
Magenta	⟷	Green	0x**00**FF**00**	Red & Blue	Green
Yellow	⟷	Blue	0x**0000**FF	Red & Green	Blue

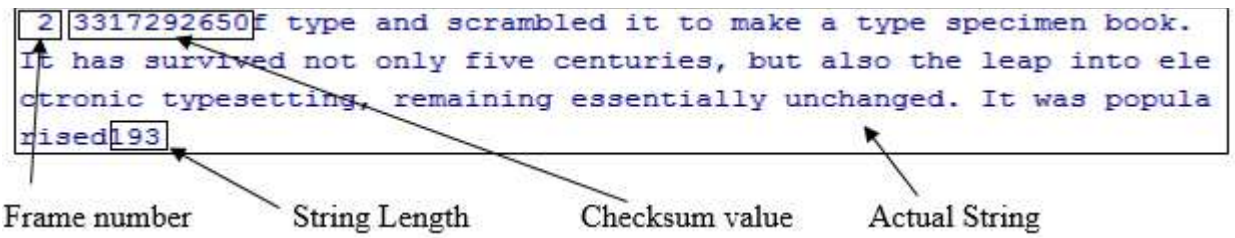


Fig. 4. Format of a string sent in one frame.

In a perfect scenario, if all the frames can be decoded from the first image, the 6th image will be decoded (i.e. after an increment of 5) followed by the 11th image and so on. This is because the next 4 images are essentially replicas of the first to provide redundancies against error in the data exchange process and they can therefore be skipped. If none of the 5 frames can be decoded correctly, *"NotFoundException"* will replace the decoded data string and the next image will be decoded.

As an example, if the first Red frame cannot be decoded due to QR code decoding exception, the next frame will be decoded. If at the 5th frame the Red frame is still unable to be decoded, *"NotFoundException"* will replace the decoded string.

The process is repeated for the Green 'channel' (5 green frames) and then the Blue 'channel' (5 blue frames).

FF represents white and *00* represents black. In QR code, data is represented by black and white pixel. For example, a cyan pixel, when converting it to grayscale image, the Red QR code will have a black pixel while the Green and Blue QR code have white pixels. Thus we are able to obtain the original pixel on the Red QR code. Likewise, for the blue pixel, when converting it to grayscale image, the Red and Green QR codes will have black pixels and Blue QR code will have white.

3) Data Integrity

CRC32 checksum is used to detect any changes to the raw data. Thus, the checksum is also sent with the actual string. Every Red, Green and Blue QR code will also have its frame number in sequence which is also being sent as part of the string to check whether it is the desired frame. The frame number is concatenated at the beginning (E.g. For the first multiplexed QR code, the Red QR code is frame number 1, Green QR code is frame number 2, Blue QR code is frame number 3, for the next multiplexed QR code, the Red QR code is frame number 4, Green QR code is frame number 5, Blue QR code is frame number 6 and so on). The checksum value is concatenated after the frame number. The actual data string is concatenated after the checksum value. The string length is also sent to check whether the decoded string has the desired length and is concatenated at the end.

Figure 4 shows an example of the format of a data frame. If there are errors, for example, say the first image was not recorded and the second image became the "first image" and the frame number starts with "4", then a checksum error situation occurs. The text file will substitute the 'lost' string with *checksumerror* and continue to decode frame 4 and so on.

The frame number is padded up to 2 characters, checksum value is padded up to 11 characters and the string length is padded up to 3 characters (since the maximum string length can only be 500 characters in 1 frame).

B. Image Transfer

The encoding and decoding process is somewhat similar to that of text file as shown in Figure 2. The difference is that the image will be converted to byte array and encoded into Base64 string before sending. At the decoder, the Base 64 string will be decoded first to retrieve the byte array and then concatenated into an image file. The CRC32 checksum will also be applied and if the results are decoded wrongly, an invalid image will be shown.

IV. Results and Discussion

The text file transfer instead of image transfer is used to test the feasibility and performance of ColorQR. This is because text file transfer allows us to evaluate the frame decoding accuracy. An LG Nexus 4 and Samsung Galaxy Note 2 are used in the testing. The Nexus 4 is equipped with a quad-core 1.5 Hz Krait CPU, 2GB of RAM, a camera capable of 1080p@30fps, and a 4.7-inch screen. The Note 2 equipped with a quad-core 1.6 GHz Cortex-A9 CPU, 2GB of RAM and the same camera specification as the Nexus 4, but it has a larger, 5.5-inch screen. The text file sent is about 300 bytes in length.

A. Decoding Accuracy and Decoding Time

The decoding accuracy is calculated based on the number of frames that can be decoded. For example, if there is a total of 3 frames for 1 flash image and only 1 frame can be decoded correctly, the accuracy is 33%. The test is first performed using the Note 2 as the sender and the Nexus 4 as the receiver. 5 transfers of the text file are captured and the average decoding accuracy is computed. The experiment is then performed with the Note 2 and Nexus 4 swapping roles.

Figure 5 shows the multiplexed QR code captured by the receivers in the test as well as the corresponding decoded grayscale QR code.

The Nexus 4 as a receiver achieves 100% decoding accuracy for all the 5 transfers. When the Note 2 is used as the receiver, the decoding accuracy is only 40%. The mean decoding time is 50.7s and 79.8s for the Note 2 and Nexus 4 respectively. Online evaluation of the two mobile devices find that the colors on the Note 2 are more saturated and colorful compared to the Nexus 4 which are more subdued. This can be seen from the brighter color in Figure 5(c) (captured from the Note 2) compared to Figure 5(a) (captured from the Nexus 4). This can account for why the Nexus 4's decoding accuracy is better when the Note 2 is the sender. Moreover, the Note 2 has a larger screen size to facilitate better video capture by the Nexus 4. Comparing Figures 5(b) and 5(d), it can be seen that the grayscale QR code decoded by the Nexus 4 has much clearer black pixels than that those by the Note 2. This explains the poor decoding accuracy of the Samsung.

In terms of CPU performance, the Note 2 is much better which therefore accounts for the longer decoding time of LG.

It can therefore be observed that decoding accuracy of ColorQR depends very much on the quality of the mobile device in terms of display quality, screen size and camera quality.

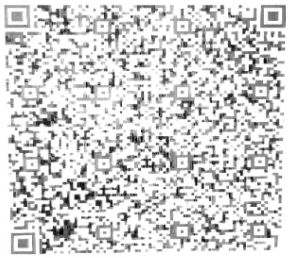
B. Encoding Speed

Five different file sizes corresponding to different number of multiplexed QR frames are encoded. The encoding speed is shown in Table VI. The results are consistent with the decoding speeds, with the Note 2 faring better than the Nexus 4

due to the former's better CPU performance. The encoding can be sped up if fewer than 5 frames are used for redundancy to recover from error in transmission.



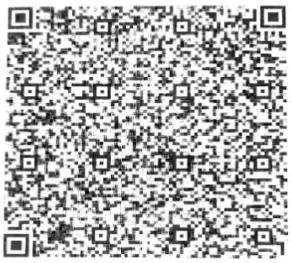
(a) Multiplexed QR code captured from the Nexus 4



(b) Grayscale QR code decoded by the Note 2



(c) Multiplexed QR code captured from the Note 2



(d) Grayscale QR code decoded by the Nexus 4

Fig. 5. Captured and decoded samples of the multiplexed QR code.

TABLE VI. AVERAGE ENCODING SPEED

File Size	1.03KB (2 frames)	2.06KB (3 frames)	3.10KB (4 frames)	4.14KB (5 frames)	5.18KB (6 frames)
Samsung Note 2	6.61s	6.72s	8.28s	9.96s	11.83s
LG Nexus 4	5.80s	8.52s	10.43s	12.52s	14.79s

C. Comparison with Related Works

With reference to the related work discussed in this paper, most of the research is innovative in the sense that the main purpose is to achieve a higher storage than existing 2D codes. However most of them are still static in nature and are intended for use in conjunction with online information. The proposed ColorQR however, does not need users to have an Internet capable device and allows users to transmit data of varying sizes between devices. It can be considered as 3D QR codes as it uses animation to flash the QR codes and hence can transmit much more data than existing QR code.

V. CONCLUSION

We presented ColorQR which presents a new way for mobile device users to exchange data in the absence of Wi-Fi, and cellular networks. Our proposed multiplexed QR code data exchange makes use of the camera of the mobile devices when users are around one another. It does not require Bluetooth discovery and pairing. The detection accuracy depends very much on the display quality, display size and camera quality of the sender and receiver devices. ColorQR does not have a limitation on the data size to be transmitted unlike other related works.

REFERENCES

[1] S. McClain, "What Are the Limitations of Bluetooth? | eHow", *eHow*, 2010. [Online]. Available: http://www.ehow.com/list_6495445_limitations-bluetooth_.html. [Accessed: 13- Dec- 2014].

[2] Qrstuff.com, "QR Code Error Correction", 2011. [Online]. Available: http://www.qrstuff.com/blog/2011/12/14/qr-code-error-correction. [Accessed: 23- Jan- 2015].

[3] DigInfo TV, "Casio PicapiCamera iPhone app is the world's first to use visible light communication technology", 2012. [Online]. Available: http://www.diginfo.tv/v/12-0117-d-en.php. [Accessed: 21- Jan- 2014].

[4] N. Harish and S. Gurav, "Embedding a Large Information in QR Code Using Multiplexing Technique", *Taraksh Journal of Communications*, vol. 1, no. 1, pp. 6-9, 2014.

[5] T. Langlotz and O. Bimber, "Unsynchronized 4D Barcodes: Coding and Decoding Time-Multiplexed 2D Colorcodes", in *Third International Symposium on Visual Computing, ISVC 2007*, Lake Tahoe, NV, USA, 2007, pp. 363-374.

[6] T. Hao, R. Zhou and G. Xing, "COBRA: Color Barcode Streaming for Smartphone Systems", in *10th International Conference on Mobile Systems, Applications, and Services, MobiSys '12*, Low Wood Bay, Lake District, UK, 2012, pp. 85-98.

[7] Microsoft Research, "High Capacity Color Barcodes (HCCB)". [Online]. Available: http://research.microsoft.com/en-us/projects/hccb/. [Accessed: 05- Mar- 2015].

[8] M. Rohs, "Real-World Interaction with Camera Phones", in *Second International Symposium on Ubiquitous Computing Systems, UCS 2004, Revised Selected Papers*, Tokyo, Japan, 2004, pp. 74-89.

[9] M. Werner and M. Schönfeld, "DualCodes: Backward Compatible Multi-layer 2D-Barcodes", in *9th International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services, MobiQuitous 2012*, Beijing, China, 2012, pp. 25-36.

[10] O. Bulan, V. Monga and G. Sharma, "High capacity color barcodes using dot orientation and color separability", *Proceedings of SPIE-IS&T Electronic Imaging*, vol. 7524, p. 725417, 2009.

[11] P. André and R. Ferreira, "Colour multiplexing of quick-response (QR) codes", *Electronics Letters*, vol. 50, no. 24, pp. 1828-1830, 2014.

[12] Opencv.org, "ANDROID | OpenCV". [Online]. Available: http://opencv.org/platforms/android.html. [Accessed: 05- Mar- 2015].